

Tema 3 - LFA

Mocanu Ștefan

1 Obiective

Această temă se concentrează pe manipularea Gramaticilor Independente de Context (CFG), verificarea apartenenței unui șir la un limbaj și o introducere în construcția compilatoarelor prin analiza lexicală.

2 Definiția Formală

O gramatică independentă de context (CFG) este un tuplu $G = (V, \Sigma, R, S)$, unde:

1. V este o mulțime finită de **neterminale** (variabile).
2. Σ este o mulțime finită de **terminale** (alfabetul limbajului), $V \cap \Sigma = \emptyset$.
3. R este o mulțime finită de **reguli de producție** de forma:

$$A \rightarrow \alpha, \text{ unde } A \in V \text{ și } \alpha \in (V \cup \Sigma)^*$$

4. $S \in V$ este **simbolul de start**.

3 Reprezentarea în fișierul de input

Pentru implementare, puteți folosi un format text simplu precum:

- Prima linie: Simbolurile neterminale (separate prin spațiu).
- A doua linie: Simbolurile terminale.
- A treia linie: Simbolul de start.
- Restul liniilor: Regulile de producție sub forma $A \rightarrow \alpha$. Pentru mai multe producții ale aceluiași neterminal, puteți folosi $A \rightarrow \alpha \mid \beta$.

Exercițiul 1: Generarea cuvintelor de lungime k

Folosind definiția unei gramatici $G = (V, \Sigma, R, S)$, implementați un algoritm care să genereze toate cuvintele de lungime **exact** k din limbajul $L(G)$.

Idee de implementare: Generarea șirurilor

Problema generării tuturor cuvintelor de lungime N poate fi tratată ca o problemă de explorare a spațiului stărilor (arborele de derivare).

Ideea de bază este să pornim de la simbolul de start S și să aplicăm succesiv regulile de producție până când șirul conține doar terminale.

1. **Starea curentă:** Un șir format din terminale și neterminale (ex: aAb).
2. **Expansiune:** Se alege primul neterminal din șir și se înlocuiește cu fiecare dintre părțile drepte ale regulilor sale de producție.
3. **Condiție de oprire (Tăierea ramurilor):**
 - Dacă lungimea șirului de terminale depășește N , acea ramură este abandonată (presupunând că nu există λ -producții care scurtează șirul).
 - Dacă șirul conține doar terminale și are lungimea dorită, este adăugat în setul de rezultate.

Exercițiul 2: Transformarea în Forma Normală Chomsky

Implementați un algoritm care transformă o gramatică independentă de context oarecare într-o gramatică echivalentă în Forma Normală Chomsky.

Pașii transformării

O gramatică este în CNF dacă toate regulile sunt de forma $A \rightarrow BC$ sau $A \rightarrow a$. Pașii obligatorii sunt:
Transformarea trebuie să respecte o ordine strictă pentru a evita reintroducerea elementelor eliminate:

- **Pasul 1: START.** Se introduce $S_0 \rightarrow S$ dacă S apare în partea dreaptă a oricărei reguli.
- **Pasul 2: TERM.** Se înlocuiesc terminalele din regulile "mixte" (ex: $A \rightarrow aB$ devine $A \rightarrow X_a B$ și $X_a \rightarrow a$).
- **Pasul 3: BIN (Binarizare).** Regulile de formă $A \rightarrow B_1 B_2 \dots B_m$ cu $m > 2$ se sparg în $A \rightarrow B_1 C_1$, $C_1 \rightarrow B_2 C_2 \dots$.
- **Pasul 4: DEL (λ -eliminare).** Se identifică neterminalele anulabile. Pentru $A \rightarrow BC$, dacă B e anulabil, se adaugă $A \rightarrow C$.
- **Pasul 5: UNIT (Eliminarea unitarelor).** Pentru $A \rightarrow B$, se adaugă $A \rightarrow \alpha$ pentru toate $B \rightarrow \alpha$.

Exercițiul 3: Algoritmul CYK (Cocke-Younger-Kasami)

Implementați algoritmul CYK pentru a verifica dacă un cuvânt w aparține limbajului unei gramatici date.

Cerință și restricții

Algoritmul CYK necesită ca gramatica să fie în **Forma Normală Chomsky (CNF)**. Programul trebuie să primească la intrare gramatica și cuvântul w , afișând un mesaj de tipul "DA/NU".

CYK utilizează programarea dinamică pentru a decide dacă $w \in L(G)$. Fie n lungimea cuvântului $w = a_1 a_2 \dots a_n$.

Structura Tabelei

Se construiește o tabelă triunghiulară T , unde $T[i, j]$ reprezintă mulțimea neterminalelor care pot genera subșirul $w_{i \dots i+j-1}$ (subșirul care începe la i și are lungimea j).

Pașii Algoritmului

1. **Pasul de bază ($j = 1$):** Pentru fiecare poziție i de la 1 la n :

$$T[i, 1] = \{A \mid A \rightarrow a_i \in R\}$$

2. **Pasul recursiv ($j = 2$ până la n):** Pentru fiecare lungime j și fiecare poziție de start i :

- Se împarte subșirul în două părți de lungimi k și $j - k$ (unde $1 \leq k < j$).
- $T[i, j] = \{A \mid A \rightarrow BC \in R, B \in T[i, k], C \in T[i + k, j - k]\}$

3. **Rezultat:** Cuvântul este acceptat dacă $S \in T[1, n]$.

CYK utilizează programarea dinamică pentru a decide dacă $w \in L(G)$. Fie n lungimea cuvântului $w = a_1 a_2 \dots a_n$.

Construcția tabelii

Având în vedere că nu se va discuta la curs și seminar despre acest algoritm, puteți urmări acest video: [completare tabelă](#).

Bonus: Analizor Lexical (Lexer)

Scrieți un Lexer (tokenizator) pentru un subset al unui limbaj de programare (ex: C, Python, Pascal).

Detalii tehnice

Programul trebuie să citească un fișier sursă și să identifice următoarele tipuri de unități lexicale (tokens):

- **Keywords:** (ex: `if`, `else`, `while`, `int`, `return`).
- **Operatori:** (ex: `+`, `-`, `*`, `/`, `=`, `==`).
- **Identificatori:** (nume de variabile/funcții care respectă regulile limbajului).
- **Literali:** (numere întregi, caractere sau șiruri de caractere).

Rezultatul va fi o listă de perechi de forma `(TOKEN_TYPE, VALUE)`.

4 Notare

- 1 pct oficiu
- 4 pct ex 1
- 2 pct ex 2
- ex 3 poate fi rezolvat în mai multe moduri (se va puncta doar cel cu punctaj mai mare)
 1. 1 pct - varianta cu căutare în lista de cuvinte de la ex 1
 2. 3 pct - rezolvat normal, ca în cerință
 3. (bonus++) 5 pct - folosind un LR(1) parser [link](#)
- 2 pct bonus lexer